2025 IEEE 7th International Conference on Computing, Communication and Automation (ICCCA) | 979-8-3315-6980-8/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICCCA66364.2025.11325363

Runtime Threat Mitigation in Kubernetes Using Falco, Falcosidekick, and Kyverno: An Automated Pod Deletion Approach

Aditya Mohan Shenoy
Department of CSE
PES University
Bangalore, India
adityamohanshenoy@gmail.com

Dr Swetha P
Associatee Professor
Department of CSE
PES University
Bangalore, India
swethap@pes.edu

Prasad B Honnavali
Professor
Department of CSE
PES University
Bangalore, India
prasadhb@pes.edu

***Abstract*—Kubernetes has revolutionized the container orchestration landscape; however, its own dynamic and distributed nature creates enormous security-related problems at runtime. One of the common vectors used for exploiting, escalation of privileges, and lateral movement is unauthorized shell access in running containers. This research presents a completely automated pipeline to counter runtime threats using open-source tools and CNCF tools namely Falco, Falcosidekick, and Kyverno — to detect and counter such threats in real time.**

The proposed architecture employs Falco to monitor system calls for suspicious activity, such as invocation of shells in containers. When such activities are detected, Falcosidekick sends notifications to a custom webhook or a Kubernetes native policy controller. Subsequently, Kyverno enforces real-time security responses by following policy definitions and performing automated pod deletions. The architecture exports a policy-enforced response in real time without human intervention, thus reducing the dwell time of the threat and the impact of attacks.

The approach is demonstrated in a local Kubernetes cluster (Minikube), with integration of observability tools such as Grafana and Loki. This exercise demonstrates a robust, modular, and vendor-neutral approach to augmenting proactive runtime security for cloud native designs, in accordance with the IEEE goals of promoting resilient infrastructure and secure automation.

***Index Terms*—Kubernetes,Runtime Security,Container Orchestration,Falco,Kyverno,Intrusion Detection,Cloud Native,Pod Deletion,Threat Mitigation,CNCF**

I. Introduction

The growing adoption of Kubernetes as the foundation for cloud-native architectures has underscored the need for robust and scalable security measures. Traditional security models often fail to address the dynamic and distributed nature of Kubernetes environments, making them vulnerable to threats such as lateral movement, privilege escalation, and misconfigured access controls [1]. The configuration requirements of the containerization platform cannot fully protect against malicious actions from containers, so it is necessary to supplement the protection system with tools that could track actions in the container in real time [2]. Falco is a native cloud security tool that provides run-time security across hosts, containers, Kubernetes, and cloud environments. It leverages custom rules on Linux kernel events and other data sources through plugins, enriching event data with contextual metadata to deliver real-time alerts. Falco enables the detection of abnormal behavior, potential security threats, and compliance violations.[3]

A Swiss Army Knife is renowned for its versatility, offering multiple tools in one compact package. Similarly, while Falco provides five standard outputs for its security events—stdout, file, gRPC, shell, and HTTP—these can sometimes fall short when integrating with other components. Enter Falcosidekick: a powerful daemon designed to vastly extend Falco's output capabilities. With over 60+ integration points, Falcosidekick elevates Falco's security events, making them a cornerstone of security automation efforts. It not only enables the addition of custom fields to events, such as environment or region, enhancing event context, but also offers metrics on event occurrences[4]

Kyverno (Greek for "govern") is a cloud native policy engine. It was originally built for Kubernetes and now can also be used outside of Kubernetes clusters as a unified policy language.Kyverno allows platform engineers to automate security, compliance, and best practices validation and deliver secure self-service to application teams.[5]

A particularly common and dangerous runtime threat involves the unauthorized execution of shells (e.g., /bin/bash) within containers. This activity is indicative of privilege escalation, reverse shells, or insider attacks. However, Kubernetes lacks native runtime intrusion detection or automatic remediation capabilities, requiring either third-party solutions or manual intervention, which delays response and increases risk.

The proposed approach:Requires no custom agents or kernel modules beyond Falco and is policy-driven, automated remediation,very lightweight, extensible, and vendor-neutral.Enhances Kubernetes' security posture without relying on commercial security suites.

II. Litreature Survey

1. **Runtime Threat Detection in Kubernetes** Securing Kubernetes environments against runtime threats has become a critical area of research and industry focus. Traditional security measures, such as static analysis and image scanning, are insufficient for detecting attacks that occur after deployment. As a result, runtime threat detection tools have gained prominence for their ability to monitor system calls, file access, and network activity in real time [6].

Falco: Kernel-Level Threat Detection Falco, an open-source runtime security tool, is widely recognized for its ability to detect suspicious behavior in containerized environments by monitoring system calls using either kernel modules or eBPF. Several technical whitepapers and security audits have analyzed Falco's architecture and effectiveness [7]. These studies highlight Falco's strengths in detecting privilege escalation, shell spawning, and file system tampering [8], but also note challenges such as performance overhead and the need for custom rule tuning to minimize false positives.

Effectiveness: Falco is effective in identifying a range of attack vectors, including file access anomalies and suspicious process execution [8].

Limitations: Performance impact in high-throughput clusters and the complexity of maintaining effective detection rules [7].

Falco Sidekick: Alert Forwarding and Automation Falco Sidekick extends Falco's capabilities by forwarding alerts to various endpoints (e.g., webhooks, messaging platforms, SIEMs) and enabling automation workflows. While there are no peer-reviewed papers focused solely on Falco Sidekick, technical documentation [10] and industry case studies [9] describe its role in integrating Falco with external systems for real-time response and observability.

Integration: Facilitates automated responses by connecting Falco alerts to remediation tools and dashboards [9].

Use Cases: Supports workflows such as automated pod deletion, incident notification, and alert enrichment [10].

2. **Policy Enforcement and Automated Remediation Kyverno: Kubernetes Policy Management** Kyverno is a Kubernetes-native policy engine designed to validate, mutate, and generate resources based on customizable policies. Security audit reports and technical documentation provide in-depth analysis of Kyverno's security features, including its webhook security, RBAC enforcement, and policy-as-code approach [11]. Recent releases have enhanced its capabilities for supply chain security, compliance reporting, and operational scalability.

Security Features: Admission control, pod security standards enforcement, image verification, and automated remediation [11].

Industry Adoption: Case studies (e.g., Robinhood) demonstrate Kyverno's scalability and effectiveness in managing security policies at scale [11].

Automated Threat Mitigation Workflows Although automated remediation—such as pod deletion in response to detected threats—is discussed in technical blogs and industry guides [12], there is a lack of peer-reviewed academic research specifically analyzing the integrated use of Falco, Falco Sidekick, and Kyverno for this purpose. Existing literature highlights the potential of such workflows to reduce dwell time and limit the impact of attacks [4], but also emphasizes the need for careful configuration to avoid unintended disruptions [12].

3. **Gaps in Existing Literature** Lack of End-to-End Academic Studies: No peer-reviewed papers comprehensively evaluate the combined use of Falco, Falco Sidekick, and Kyverno for automated runtime mitigation (e.g., pod deletion).

Focus on Detection or Policy in Isolation: Most research examines detection (Falco) or policy enforcement (Kyverno) separately, rather than as part of an integrated, automated threat response pipeline.

This work shows robust technical and industry support for runtime attack detection and automated policy enforcement in Kubernetes, with Falco, Falco Sidekick, and Kyverno creating a robust toolchain. Yet, the lack of large-scale academic research on their combined application to automatically delete pods reveals an important gap that my paper attempts to fill.

III. Methodology

This section outlines the design and implementation of a Kubernetes-native threat mitigation pipeline that automatically deletes compromised pods upon detection of suspicious runtime behavior mainly shell spawning. This system integrates three primary open-source components: Falco for runtime detection, Falcosidekick for alert forwarding, and Kyverno for policy-based remediation.

A. System Architecture

This system architecture consists of the following components deployed within a local Kubernetes cluster using Minikube:Falco runs as a DaemonSet on all nodes, monitoring kernel-level system calls in real time.Falcosidekick acts as an alert forwarder, receiving JSON-formatted alerts from Falco and sending them to configured endpoints (e.g., webhooks, logging systems).A custom webhook receiver listens for Falcosidekick alerts and applies a Kubernetes label (e.g., suspicious=true) to the offending pod.Kyverno, a Kubernetes-native policy engine, enforces a policy that automatically deletes pods labeled as suspicious.

This architecture ensures a fully automated pipeline from threat detection to remediation without requiring human intervention.

B. Pre-Requisites

A few things have to be set-up before diving into the main system architecture, starting with virstualization support enabled in your operating system which helps falco,loki and grafna getting access of the system calls , the following commands are pertainig to Ubuntu 22.04 (latest) operating system.

This is a command to check if Virtualization is enabled or not , if its not enabled you have to enable it through BIOS/UEFI

egrep -q 'vmx—svm' /proc/cpuinfo && echo "Virtualization supported" or echo "Not supported"

Next step is to install and setup **Docker**:Docker is an open platform for developing, shipping, and running applications. Docker enables you to separate your applications from your infrastructure so you can deliver software quickly. With Docker, you can manage your infrastructure in the same ways you manage your applications. By taking advantage of Docker's methodologies for shipping, testing, and deploying code, you can significantly reduce the delay between writing code and running it in production.[13]Here Docker is used as container runtime and driver for minikube Next is to install and setup Helm for deploying Falco, Sidekick and Loki **curl https://raw.githubusercontent.com/helm/helm/main /scripts/get-helm-3 — bash**

C. Minikube Environment Setup

Minikube is an open-source tool that allows you to run a local Kubernetes cluster on your personal machine. It is designed to make learning, developing, and testing Kubernetes applications simple and accessible without the need to set up a complex, multi-node production cluster.[14]

Here we will be using Minikube to locally demonstrate kubernetes clusters.

1) first we begin with installing all the dependencies

```
sudo apt update
sudo apt install -y curl wget apt-transport-https
```

Fig. 1. Minikube Dependencies

2) install minikube

```
curl -LO https://github.com/kubernetes/minikube/releases/latest/download/minikube-linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube && rm minikube-linux-amd64
```

Fig. 2. minikube installation

3) install kubectl

4) start Minikube

D. Setting up Falco and Adding Custom Rule

Falco is installed in the Kubernetes cluster to monitor system-level activity and detect suspicious behavior using custom rules. Below is the complete setup process, including integration of a custom detection rule.

1) Step 1: Installing Falco via Helm: Falco can be installed from the official Helm chart repository:

```
helm repo add falcosecurity https://
    falcosecurity.github.io/charts
helm repo update
helm install falco falcosecurity/falco --
    namespace falco --create-namespace
```

2) Step 2: Creating a Custom Rule: Rather than modifying the core `rules.yaml` directly—which would require elevated permissions and is not recommended—we use a custom `ConfigMap` to inject our rule into the running Falco instance.

First, create a file named `custom-rules.yaml` containing the desired detection rule:

```
- rule: Terminal shell in container
  desc: Detects shell spawned inside a
      container
  condition: container.id != host and proc.
      name in (bash, sh, zsh)
  output: "Shell spawned inside container (
      user=%user.name command=%proc.cmdline
      container=%container.name)"
  priority: CRITICAL
  tags: ["container", "shell"]
```

This rule triggers an alert whenever a terminal shell (bash, sh, or zsh) is executed inside a container.

3) Step 3: Creating the ConfigMap: Create a Kubernetes ConfigMap from the YAML file using the following command:

```
kubectl create configmap falco-custom-rules \
  --from-file=custom-rules.yaml \
  --namespace=falco
```

4) Step 4: Mounting the ConfigMap into Falco: Patch the existing Falco installation by upgrading it to include the custom ConfigMap. This can be done using the `--set-file` flag with Helm:

```
helm upgrade falco falcosecurity/falco \
  --namespace falco \
  --set-file customRules.configMap=falco-
      custom-rules
```

Once upgraded, Falco automatically picks up the custom rule and begins monitoring for any matching activity. In our case, this enables Falco to detect when a shell is spawned inside a container—forming the basis for our runtime threat mitigation workflow.

E. Alert Forwarding with Falcosidekick

Falcosidekick acts as a bridge between Falco and external systems. Once Falco detects an anomaly and generates an alert, Falcosidekick receives that alert and forwards it to configured endpoints, including webhooks and Grafana Loki.

1) Installation and Configuration: To install Falcosidekick with a webhook configuration, run the following Helm command:

```
helm install falcosidekick falcosecurity/
    falcosidekick \
  --namespace falco --create-namespace \
  --set config.webhook.address=http://falco-
      webhook.falco.svc.cluster.local:5000/
```

This command installs Falcosidekick in the `falco` namespace and configures it to send alerts to a custom webhook server listening at `http://falco-webhook.falco.svc.cluster.local:5000/`.

Note: The forwarding server address may vary depending on your environment. Replace it with your actual webhook service endpoint within the cluster.

2) Deployment Verification: After installation, Kubernetes will create pods for both Falco and Falcosidekick. To verify that the webhook integration is working, inspect the logs of the Falcosidekick pod. A successful alert forwarding log may look like:

```
{
  "output": "Shell spawned inside container (
      user=root command=bash container=ubuntu-
      attacker)",
  "priority": "CRITICAL",
  "output_fields": {
    "k8s.pod.name": "ubuntu-attacker",
    "k8s.namespace.name": "default"
  },
  "rule": "Terminal shell in container"
}
```

This alert payload is received by the webhook server and is used to trigger a response—such as labeling or deleting the pod.

3) Webhook Integration with Flask: In our setup, the alert payload is forwarded in JSON format to a lightweight Flask-based webhook server. Flask was chosen for its simplicity and compatibility with Falco and Falcosidekick.

The alert payload, as shown above, is parsed by the Flask application and used to label the suspicious pod for further remediation by Kyverno.

This completes the setup and configuration of Falcosidekick for real-time alert forwarding within the Kubernetes environment.

Fig. 3. alert payload

here ”ubuntu-atacker is the custom local pod created and in which it is placed in the default namespace in the k8’s minikube cluster

F. Custom Webhook Server

This implementation is a lightweight webhook server using Python’s Flask framework to receive alerts from Falcosidekick and label the offending pod. This label enables Kyverno to detect suspicious pods and take remediation action.

1) Flask Webhook Code: The webhook server listens for incoming JSON payloads from Falcosidekick. Upon receiving an alert, it extracts the pod name and namespace, then applies a Kubernetes label `suspicious=true` to the pod using `kubectl`:

```
from flask import Flask, request
import subprocess

app = Flask(__name__)

@app.route('/', methods=['POST'])
def alert():
    data = request.json
    pod = data['output_fields']['k8s.pod.name'
        ]
    ns = data['output_fields']['k8s.namespace.
        name']
    subprocess.call(["kubectl", "label", "pod"
        , pod, "suspicious=true", f"-n{ns}", "
        --overwrite"])
    return "OK", 200

app.run(host='0.0.0.0', port=5000)
```

2) Containerizing the Webhook: To deploy this webhook inside a Kubernetes cluster, we containerized it using Docker. The Dockerfile used for this process is shown in Fig. 4.

```
FROM python:3.10-slim
WORKDIR /app
COPY webhook.py /app/
RUN pip install flask
CMD ["python", "webhook.py"]
```

Fig. 4. Dockerfile for webhook container

3) Deploying the Webhook in Kubernetes: Next, we created a Kubernetes `Deployment` to run the webhook container. This is shown in Fig. 5.

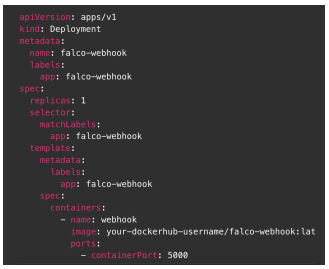

Fig. 5. Kubernetes deployment manifest for webhook

Note: Be sure to replace the container image name in the deployment specification with your actual DockerHub username or image path.

4) Exposing the Webhook Service: In order to allow Falcosidekick to communicate with the Flask webhook, we created a Kubernetes `Service` to expose the deployment within the cluster (Fig. 6).

5) Building and Deploying the Webhook: Once the Dockerfile and manifests are ready, we built the Docker image and deployed it to the Kubernetes cluster using standard commands, as illustrated in Fig. 7.

This setup allows Falcosidekick to reliably reach the webhook inside the Kubernetes cluster, ensuring alerts are converted into actionable pod labels for Kyverno to act upon.

6) Flask Code Recap: A final recap of the Flask server logic is shown in Fig. 8 for clarity.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: falco-webhook
spec:
  selector:
    app: falco-webhook
  ports:
    - protocol: TCP
      port: 80
      targetPort: 5000
```

Fig. 6. Exposing the webhook as a Kubernetes service

```
kubectl apply -f webhook-deployment.yaml
kubectl apply -f webhook-service.yaml
```

Fig. 7. Building and applying the webhook to the Kubernetes cluster

```python
from flask import Flask, request
import subprocess

app = Flask(__name__)

@app.route('/', methods=['POST'])
def alert():
    data = request.json
    pod = data['output_fields']['k8s.pod.name']
    ns = data['output_fields']['k8s.namespace.name']
    subprocess.call(["kubectl", "label", "pod", pod, "susp
    return "OK", 200

app.run(host='0.0.0.0', port=5000)
```

Fig. 8. Flask code to receive alerts and label pods

This server was deployed in the cluster as a Kubernetes Deployment and exposed via a Service so that Falcosidekick could reach it.

G. Kyverno Policy for Auto Deletion

Kyverno works on policy babsis and now we have created a custom policy to address the alert recieved from falco sidekick and delete the pod.

the ClusterPolicy looks something like this:

```yaml
apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata:
  name: delete-suspicious-pods
spec:
  rules:
    - name: delete-pods-on-label
      match:
        any:
          - resources:
              kinds:
                - Pod
              selector:
                matchLabels:
                  suspicious: "true"
      validate:
        message: "Suspicious pod detected."
        deny: {}
```

Fig. 9. ClusterPolicy(kvyerno

then to apply this policy on the locally running k8 cluster : kubectl apply -f delete-suspicious-pods.yaml

Kyverno enforces this policy automatically whenever a pod is labeled suspicious=true, resulting in its deletion.

Now that our complete setup is ready we proceed to test it

H. Testing and workflow execution

As mentioned earlier we have tested this locally , we first created sample test pod name ”ubuntu-atacker” and executed shell inside it.

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: ubuntu-attacker
spec:
  containers:
    - name: ubuntu
      image: ubuntu
      command: ["sleep"]
      args: ["3600"]
```

Fig. 10. sample pod

to execute a shell/spawn a shell inside this pod we simply executed this comand :kubectl exec -it ubuntu-attacker – bash

I. Result

This pipeline for security monitoring was successfully deployed on a Kubernetes cluster utilizing open-source tooling such as Falco, Falcosidekick, and a bespoke webhook service. The intention was to pick up and respond to container-level threats in real time, namely unauthorized shell access within pods.

Falco was run as a DaemonSet on all nodes to observe system calls at the kernel level. A specific Falco rule was configured to detect terminal shells that were spawned within containers. As soon as Falco detected a match, it created structured security alerts and sent them to Falcosidekick, acting as an event router.

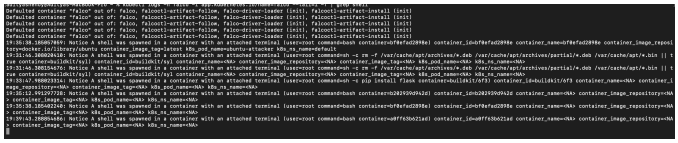

Fig. 11. Detection

Falcosidekick was set to send these alerts to a Flask-based HTTP webhook server deployed inside the cluster. This webhook extracted pod metadata from the received alert payload and initiated the deletion of the malicious pod via kubectl. As a simulation, a pod (ubuntu-attacker) was created and reached through an interactive shell. The pipeline worked as expected, recognizing the shell activity and removing the pod automatically.

The experimental findings ratified that combining Falco, Falcosidekick, and a webhook mechanism(kvyerno) built specifically can facilitate near real-time detection and remediation of container-based attacks in Kubernetes environments.

Fig. 12. executing shell

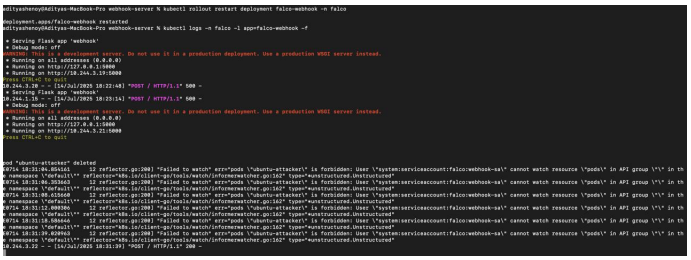

Fig. 13. deleted

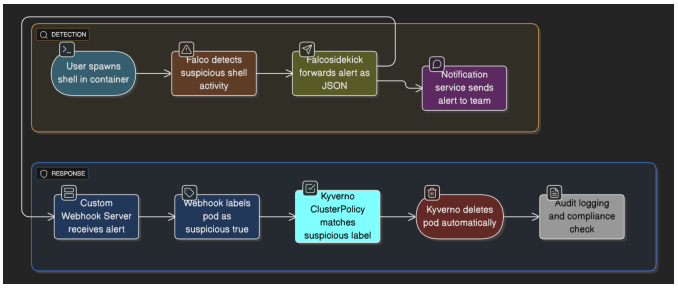

Fig. 14. Process Flow

IV. Conclusion and Future Work

In this work, we presented a Kubernetes-native, automated runtime threat mitigation workflow that combines open-source tools—Falco, Falcosidekick, a custom webhook, and Kyverno—to detect and respond to suspicious activity in real-time. By deploying this stack on a local Minikube environment, we demonstrated a fully functional pipeline that detects shell access inside containers and automatically deletes compromised pods.

This solution leverages Falco's kernel-level syscall monitoring to identify threats, Falcosidekick's alert forwarding capabilities, a lightweight Flask webhook to label offending pods, and Kyverno's policy engine to enforce automated remediation. This modular design allows for scalable, cloud-native defense mechanisms without relying on external proprietary platforms.

The experimental setup confirms the effectiveness of this integration in reducing mean time to detection (MTTD) and response (MTTR) during container compromise events. The architecture is simple, reproducible, and compatible with most Kubernetes distributions, making it suitable for adoption in both academic research and enterprise DevSecOps pipelines.

A. Future Work

While the proposed system is effective for detecting and mitigating shell-based threats, several areas remain open for future exploration:

1)Extending Detection Coverage: Future work will focus on incorporating additional detection rules to handle lateral movement, privilege escalation, file system tampering, and network-based anomalies.

2)Policy Granularity: Kyverno policies can be made more context-aware, enabling differentiated responses (e.g., isolate pod instead of deletion, notify SOC teams, trigger canary redeployments).

3)Integration with SIEMs and SOAR Platforms: Integrating this workflow with platforms like Splunk, ELK, or Wazuh for centralized alert correlation and forensic analysis would enhance observability and incident triage.

4)Conformance and Compliance Audits: Mapping Falco and Kyverno events to frameworks like MITRE ATT&CK and NIST 800-190 can make the solution viable for regulated industries.

5)AI-Driven Policy Tuning: Exploring ML-based tuning for Falco rules and Kyverno policy recommendations could further reduce false positives and improve resilience against novel threats.

References

[1] R. F. dos Santos, "Applying Zero Trust to Kubernetes Clusters," *ARIS2-Advanced Research on Information Systems Security*, vol. 5, no. 1, pp. 57–71, 2025. [Online]. Available: https://scholar.google.com/scholar?hl=en&as_sdt=0\%2C5&q=Applying+Zero+Trust+to+Kubernetes+Clusters

[2] K. German and O. Ponomareva, "An Overview of Container Security in a Kubernetes Cluster," in *Proc. 2023 IEEE Ural-Siberian Conf. on Biomedical Engineering, Radioelectronics and Information Technology (USBEREIT)*, Yekaterinburg, Russia, 2023, pp. 283–285, doi: 10.1109/USBEREIT58508.2023.10158865.

[3] Falco Project, "Falco: Cloud Native Runtime Security," 2023. [Online]. Available: https://falco.org/

[4] Cloud Native Computing Foundation, "Cloud Native Live: Falcosidekick – The Swiss Army Knife for Cloud Native Security & Observability," 2023. [Online]. Available: https://www.cncf.io/online-programs/cloud-native-live-falcosidekick-the-swiss-army-knife-for-cloud-native-security-observability/

[5] Kyverno Project, "Introduction to Kyverno," 2023. [Online]. Available: https://kyverno.io/docs/introduction/

[6] Cloud Native Computing Foundation, "Kubernetes Runtime Security," CNCF Whitepaper, 2023.

[7] Quarkslab, "Security Audit of Falco," CNCF, 2023.

[8] D. Kumar, "Evaluating Falco's Ability to Detect Lateral Movement in Kubernetes," M.S. thesis, Radboud University, 2025.

[9] Mercari Engineering, "Threat Detection in Kubernetes with Falco and Sysdig Secure," Mercari, 2025.

[10] CNCF, "Falco Sidekick Documentation," GitHub. [Online]. Available: https://github.com/falcosecurity/falcosidekick

[11] Ada Logics Ltd, "Kyverno 2023 Security Audit Report," CNCF, 2023.

[12] Various Authors, "Kubernetes Response Engine: Falcosidekick + Tekton/Fission/Knative," Technical Blog, 2023.

[13] Docker Inc., "Docker Overview," 2023. [Online]. Available: https://docs.docker.com/get-started/docker-overview/

[14] Kubernetes SIGs, "Minikube Documentation," 2023. [Online]. Available: https://minikube.sigs.k8s.io/docs/